Modern Cryptography and AI/ML Security

LECTURE 30

Differential Privacy

IISER THIRUVANANTHAPURAM

Course ID: DSC 4110/6111 **Instructor:** Dr. Laltu Sardar **Date:** October 26, 2026

1 Introduction: What Privacy Does Differential Privacy Protect?

1.1 The privacy objective

Organizations collect data about people and use it to train machine-learning models. A model may then be given to another organization, a server, or end users. Differential privacy (DP) studies how to limit what the released model reveals about any one person's contribution to the training dataset.

Consider an owner who collects records and trains a model. The owner gives the model to a separate server. The server is curious or malicious and tries to determine whether a particular person's record was used in training. In this setting, the server is the adversary. More generally, the adversary may be any party other than the trusted data owner who can observe the released model or its outputs and attempt to infer information about a training-data contributor.

The central objective is therefore:

> *Limit the additional information that the released model can reveal about whether a particular individual's data contributed to training.*

This lecture focuses on training-data privacy. It does not assume that DP hides an inference query from the server, nor does it aim to keep the model's parameters secret. Query privacy and model confidentiality are separate goals that generally require other mechanisms, such as secure computation or homomorphic encryption.

1.2 Practical applications

Differential privacy is used when organizations need to publish useful statistics or models while limiting the effect of individual records. Important examples include privacy-preserving population statistics, such as the US Census Bureau's use of differential privacy for the 2020 Census; telemetry and usage measurement, including local privacy mechanisms developed for collecting aggregate statistics; and machine-learning systems that use DP-SGD to limit the influence of individual training examples. In each case, the system must choose a privacy definition, a privacy budget, and a noise mechanism appropriate to the data and intended release. DP does not mean that the data is never accessed internally: in central DP, a trusted curator may process the original data before releasing a protected output.

2 What Can Go Wrong Without Privacy?

If a model is trained and released without a suitable privacy guarantee, its outputs may reveal information about individual training records. Three illustrative attacks are the following.

2.1 Membership inference

The attacker wants to decide whether a target person's record was included in training. For example, an attacker knows a patient's record and queries a disease-prediction model. The attacker uses confidence scores, prediction behavior, or other available information to estimate whether that record was part of the training set. Query access is common in this attack, although some attacks use model parameters or other information instead.

2.2 Model inversion

The attacker attempts to reconstruct sensitive features or representative information about training examples from model outputs or parameters. For example, repeated observations of a model's responses may help an attacker infer sensitive characteristics associated with the training data. The feasibility of such an attack depends on the model, the information exposed, and the attacker's background knowledge.

2.3 Attribute inference

The attacker knows some facts about an individual but tries to infer a missing sensitive attribute. For example, the attacker knows a patient's age and some non-sensitive characteristics and uses model behavior or auxiliary information to estimate a disease status. This does not necessarily require deciding whether the exact record appeared in training.

These attacks are not guaranteed to succeed, and DP does not prevent every inference about a person. Instead, DP limits how much the distribution of a released output can change because one person's record is included rather than absent. Information that is already public or can be inferred from other sources may remain inferable.

The motivating questions are therefore:

1. How can privacy be defined formally?
2. How can the strength of a privacy guarantee be measured?
3. How can a system achieve the guarantee?

A common way to achieve DP is to introduce carefully calibrated randomness, often described as adding noise. Noise creates a trade-off: more noise usually gives stronger privacy but can reduce the accuracy or usefulness of the released result. The goal is not to add arbitrary noise, but to add enough noise to satisfy a defined guarantee while preserving as much utility as possible.

3 Neighboring Datasets and the Formal Definition

3.1 Neighboring datasets

Let D denote a dataset of records. Two datasets D and D' are called *neighbors* if they differ in the contribution of one individual. The exact definition must be stated because it affects sensitivity and noise calibration.

Two common definitions are:

- **Add/remove adjacency:** one dataset is obtained from the other by adding or removing one individual's record.
- **Replace-one adjacency:** one individual's record is replaced by another record.

For example, if D contains 100 patient records, a neighboring dataset under add/remove adjacency may contain the same 99 records after one patient's record is removed. The privacy guarantee compares the mechanism's behavior on these neighboring datasets.

3.2 Formal definition of differential privacy

A randomized mechanism M satisfies (ε, δ)-differential privacy if, for every pair of neighboring datasets D, D' and every measurable set of possible outputs S,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta.$$

The inequality must hold for both orderings of every neighboring pair. Here, M is a randomized algorithm, not necessarily a count function. In a machine-learning application, M may be the complete training procedure and its output may be the trained model. The set S is any measurable event in the output space, such as the set of models that return a particular prediction. It is not the adversary; it describes an event whose probability is compared across neighboring datasets.

3.3 Interpreting ε and δ

The parameter ε controls the multiplicative difference between output probabilities. Smaller ε generally means a stronger privacy guarantee, while larger ε permits the output distributions to be more distinguishable. The quantity e^{ε} is the multiplicative factor in the definition.

The parameter δ is an additive allowance in approximate DP. When $\delta = 0$, the guarantee is called pure ε-DP. When $\delta > 0$, it is called approximate (ε, δ)-DP. A meaningful approximate-DP guarantee uses a small δ; if δ is close to one, the guarantee can be weak. By convention, $0 \leq \delta < 1$.

If two mechanisms have parameters $(\varepsilon_1, \delta_1)$ and $(\varepsilon_2, \delta_2)$, and both $\varepsilon_1 \leq \varepsilon_2$ and $\delta_1 \leq \delta_2$, then the first guarantee is at least as strong under this coordinate-wise comparison. If one mechanism has a smaller ε but a larger δ, neither pair is automatically stronger by this simple comparison. For example, $(1, 10^{-3})$ and $(2, 10^{-6})$ trade off the two parameters, so the pairs are not ordered coordinate-wise. A meaningful comparison requires a common privacy requirement and, where relevant, analysis of the mechanisms' privacy-loss behavior. The parameters describe a guarantee, not a direct measurement of the exact amount of information an attacker will learn.

3.4 Sensitivity: how much can the output change?

For a function f on datasets, its global sensitivity measures the largest change in its output over neighboring datasets. For a vector-valued function and a chosen norm $\|\cdot\|$,

$$\Delta f = \max_{D \sim D'} \|f(D) - f(D')\|.$$

For example, under add/remove adjacency, the count of records with a particular property has sensitivity one: adding or removing one record changes the count by at most one. Sensitivity helps determine the noise scale needed by a mechanism. It is important to distinguish a formal upper bound from an empirical estimate: testing a few neighboring datasets does not prove global sensitivity.

4 Where Can Noise Be Added?

Consider a typical workflow:

$$\text{Collect data} \longrightarrow \text{Train model} \longrightarrow \text{Outsource or release model.}$$

There are three common locations for introducing randomness to protect training-data privacy:

1. **Input perturbation:** perturb the collected records before training starts.
2. **Output perturbation:** train first, then perturb the trained model parameters before releasing the model.
3. **Training-time perturbation:** add calibrated noise to gradients or updates while training, as in DP-SGD.

These are implementation approaches, not three different definitions of DP. Adding noise at any location does not automatically provide DP; the mechanism, adjacency relation, sensitivity, and privacy accounting must support the claimed guarantee.

5 Input Perturbation: Noise Before Training

5.1 Motivation and use case

Input perturbation modifies each record before it is used for training. It is especially relevant when the entity collecting data is different from the entity training the model. For example, a collector may receive randomized records from users and then provide those records to a separate model trainer. The trainer does not directly receive the users' original values.

A common design is local differential privacy (LDP), in which each user's device randomizes the data before transmission. If the collector itself is not trusted with raw values, this is an important distinction

from central DP, where a trusted curator sees the raw data and protects the output later. Input perturbation can reduce data quality, and a valid privacy guarantee requires a mechanism designed for the input domain and the privacy unit.

5.2 Generic algorithm

Let $D = \{z_1, \ldots, z_n\}$ be the collected dataset. For each record, apply a randomized mechanism R with an appropriate privacy parameter, then train on the perturbed dataset.

Algorithm: InputPerturbation
Input: Dataset $D = \{z_1, \ldots, z_n\}$; per-record mechanism R; privacy parameters.
Output: Trained model $\widetilde{\theta}$.

1. Initialize an empty dataset $\widetilde{D}$.
2. For each record $z_i \in D$, sample randomness and compute $\widetilde{z}_i \leftarrow R(z_i)$.
3. Add $\widetilde{z}_i$ to $\widetilde{D}$.
4. Train the model: $\widetilde{\theta} \leftarrow A(\widetilde{D})$.
5. Return $\widetilde{\theta}$.

The distribution used by R depends on the data type and privacy goal. If each record is protected by an ε-LDP mechanism, subsequent computation on the randomized records is post-processing and does not weaken that local guarantee. If multiple attributes are randomized, privacy composition must be considered. Noise independently added to raw features is not automatically a record-level DP guarantee.

5.3 Common distributions and mechanisms

Laplace distribution. For a numerical query or bounded scalar, Laplace noise is often used when a pure ε-DP mechanism can be specified. For a query f with global sensitivity Δf, the standard Laplace mechanism releases

$$M(D) = f(D) + Z, \qquad Z \sim \mathrm{Lap}\left(0, \frac{\Delta f}{\varepsilon}\right).$$

This formula applies to a query output; applying Laplace noise to each raw feature requires a separately specified input mechanism and sensitivity/domain assumptions.

Gaussian distribution. Gaussian noise is commonly used for vector-valued mechanisms that target approximate (ε, δ)-DP. Its standard deviation must be calibrated to an appropriate L_2 sensitivity and the desired privacy parameters. For raw-data perturbation, the mechanism must also account for the data domain and whether privacy is required locally or centrally.

Discrete Gaussian distribution. The discrete Gaussian is supported on integers and is useful when values are integer-valued or when integer arithmetic is required, including some privacy-preserving statistical and distributed-computation settings. It is not automatically the best choice for ordinary real-valued features. Its parameters must be calibrated to the mechanism's sensitivity and target guarantee.

Which should be used? There is no universally best distribution. Laplace is a standard choice for pure-DP scalar numerical queries; Gaussian is widely used for approximate DP and vector-valued mechanisms; discrete Gaussian is suitable for integer-valued outputs or computations that require discrete noise. For categorical records, randomized response or another domain-specific mechanism may be more appropriate than adding numeric noise.

6 Output Perturbation: Noise After Training

6.1 Motivation and use case

In output perturbation, the trainer first runs a training algorithm on the original dataset and then perturbs the resulting model parameters before releasing the model. This is useful when the data owner or trusted trainer can access the original dataset but wants the model recipient to receive a privacy-protected model.

Let $A(D) = \theta_D \in \mathbb{R}^d$ be the trained parameter vector. The output is

$$\widetilde{\theta} = A(D) + Z.$$

The noise must be calibrated to a valid upper bound on the sensitivity of the entire training algorithm:

$$\Delta_2(A) = \max_{D \sim D'} \|A(D) - A(D')\|_2.$$

This is a vector-level quantity, not a separate sensitivity that must be computed for each coordinate. In practice, one usually proves a theoretical upper bound from assumptions on the training algorithm rather than enumerating all neighboring datasets. Such a bound may be difficult to establish for arbitrary deep neural networks.

6.2 Algorithm

Algorithm: OutputPerturbation
Input: Dataset D; training algorithm A; privacy parameter $\varepsilon > 0$.
Output: Perturbed model parameters $\widetilde{\theta}$.

1. Train the model: $\theta \leftarrow A(D)$.
2. Establish a valid theoretical upper bound Δ such that
$$\Delta \geq \max_{D \sim D'} \|A(D) - A(D')\|_2.$$
3. Sample a noise vector $Z \in \mathbb{R}^d$ from a distribution calibrated to Δ and the target privacy parameters.
4. Compute $\widetilde{\theta} \leftarrow \theta + Z$.
5. Release $\widetilde{\theta}$, not θ.

For example, a vector-valued Laplace mechanism with density

$$p(z) = c \exp\left(-\frac{\varepsilon \|z\|_2}{\Delta}\right)$$

for normalization constant c gives pure ε-DP when Δ is a valid L_2 sensitivity bound. This is a joint multivariate distribution; it should not be confused with independently sampling each coordinate from a scalar Laplace distribution. Gaussian noise can also be used with appropriate calibration for approximate DP.

6.3 Choice of noise

- **Laplace-type vector noise:** appropriate when pure ε-DP is desired and the algorithm's L_2 sensitivity can be bounded.
- **Gaussian noise:** appropriate when approximate (ε, δ)-DP is acceptable and the sensitivity bound supports Gaussian calibration.
- **Discrete Gaussian:** potentially useful for integer-valued model representations or integer computations, but not the usual default for real-valued neural-network parameters.

The key challenge is often not sampling the noise but proving a useful sensitivity bound for the training algorithm. A well-known reference for output perturbation is Chaudhuri, Monteleoni, and Sarwate, "Differentially Private Empirical Risk Minimization," JMLR, 2011. Their results rely on assumptions on the loss and regularization and should not be taken as a guarantee for every model-training procedure.

7 Training-Time Perturbation: Stochastic Gradient Descent and DP-SGD

7.1 When is it useful?

Training-time perturbation is widely used for machine learning, especially when directly bounding the sensitivity of the final trained parameters is difficult. DP-SGD limits each training example's influence by clipping its gradient and then adds calibrated noise to the aggregated gradient. The model is updated using the noisy gradient throughout training. Privacy accounting combines the effects of sampling, noise, and repeated updates.

7.2 Stochastic Gradient Descent

Let the training dataset be

$$D = \{z_1, z_2, \ldots, z_n\},$$

where z_i denotes one training example. Let $\theta \in \mathbb{R}^d$ be the model parameters and $\ell(\theta; z_i)$ be the loss on example z_i. The training objective is to minimize the average loss:

$$L(\theta; D) = \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; z_i).$$

Gradient descent updates the parameters using the gradient of the objective. However, computing this gradient requires processing the entire dataset at each iteration. Stochastic gradient descent (SGD) reduces this cost by selecting a random example or mini-batch and computing the gradient using only the selected data.

For a mini-batch B_t, the gradient estimate is

$$g_t = \frac{1}{|B_t|} \sum_{z_i \in B_t} \nabla_\theta \ell(\theta_t; z_i).$$

The model parameters are updated as

$$\theta_{t+1} = \theta_t - \eta_t g_t,$$

where $\eta_t > 0$ is the learning rate. This process is repeated until the specified number of training steps is completed or a stopping criterion is satisfied.

Ordinary SGD does not guarantee differential privacy. An individual training example can have a significant influence on the gradient and, consequently, on the final model. Differentially private SGD (DP-SGD) limits this influence by clipping individual gradients and adding random noise to their aggregate.

7.3 Differentially Private SGD

DP-SGD modifies ordinary SGD in two steps: gradient clipping and Gaussian noise addition. First, each per-example gradient is clipped to have a maximum L_2-norm $C > 0$. For

$$g_i = \nabla_\theta \ell(\theta_t; z_i),$$

the clipped gradient is

$$\bar{g}_i = g_i \min\left(1, \frac{C}{\|g_i\|_2}\right).$$

Thus, $\|\bar{g}_i\|_2 \leq C$, which bounds the contribution of each example to the gradient aggregate.

Next, independent Gaussian noise is added to the sum of clipped gradients. For a mini-batch B_t of fixed size B, define

$$Z_t \sim \mathcal{N}(0, \sigma^2 C^2 I_d),$$

where d is the number of model parameters, I_d is the d-dimensional identity matrix, and $\sigma > 0$ is the noise multiplier. The noisy gradient is

$$\widetilde{g}_t = \frac{1}{B}\left(\sum_{z_i \in B_t} \bar{g}_i + Z_t\right).$$

The model is then updated using

$$\theta_{t+1} = \theta_t - \eta_t \widetilde{g}_t.$$

Clipping bounds each example's contribution, while Gaussian noise reduces the information revealed by the aggregate. The privacy guarantee depends on the sampling scheme, clipping bound, noise multiplier, number of steps, and privacy accounting method. These parameters must be selected together to achieve a target (ε, δ)-differential privacy guarantee.

7.4 Algorithm for DP-SGD

Algorithm 1 describes the training procedure. It assumes fixed-size mini-batches and uses a privacy accountant to evaluate the privacy loss accumulated over all training steps.

Algorithm 1 Differentially Private Stochastic Gradient Descent

Require: Dataset D; initial parameters θ_0; learning rates $\{\eta_t\}_{t=0}^{T-1}$; batch size B; clipping bound C; noise multiplier σ; number of steps T.

Ensure: Trained parameters θ_T and the reported privacy guarantee.

1: Select a sampling scheme and a privacy accountant.
2: Choose σ and T to meet the target privacy budget.
3: **for** $t = 0, 1, \ldots, T-1$ **do**
4: Sample a mini-batch $B_t \subseteq D$ of size B.
5: **for** each $z_i \in B_t$ **do**
6: Compute $g_i \leftarrow \nabla_\theta \ell(\theta_t; z_i)$.
7: Clip the gradient:

$$\bar{g}_i \leftarrow g_i \min\left(1, \frac{C}{\|g_i\|_2}\right).$$

8: **end for**
9: Sample $Z_t \sim \mathcal{N}(0, \sigma^2 C^2 I_d)$.
10: Compute the noisy gradient:

$$\widetilde{g}_t \leftarrow \frac{1}{B}\left(\sum_{z_i \in B_t} \bar{g}_i + Z_t\right).$$

11: Update parameters:

$$\theta_{t+1} \leftarrow \theta_t - \eta_t \widetilde{g}_t.$$

12: **end for**
13: Compute the total privacy cost using the specified sampling scheme and privacy accountant.
14: Verify that the resulting guarantee meets the target (ε, δ).
15: **return** θ_T and the privacy guarantee.

The algorithm adds noise to the aggregate of clipped gradients, not directly to each model parameter. The noise Z_t is a random vector whose coordinates are sampled jointly according to the specified multivariate Gaussian distribution. The division by B scales both the gradient sum and the added noise to obtain the noisy average gradient.

The privacy accountant tracks the cumulative privacy loss across training steps. If the resulting privacy guarantee exceeds the target budget, the training configuration must be adjusted and evaluated again. Subject to the required assumptions and correct accounting, DP-SGD provides a formal privacy guarantee for the training records. Its main practical trade-off is between privacy and model utility: stronger privacy generally requires more noise, which can reduce model accuracy.

8 Comparison of the Three Approaches

The approaches are not interchangeable. Input perturbation changes the data seen by the trainer. Output perturbation changes the model released after training. DP-SGD changes the training updates. Each approach must be analyzed under a clearly stated privacy unit and adjacency relation. For example, record-level DP and user-level DP can require different mechanisms and noise scales.

Approach	Where noise is added	Main motivation	Key issue
Input perturbation	Individual records before training	Useful when the collector should not receive raw values; supports local-DP designs	Perturbed records may be less useful; privacy composition across fields must be handled
Output perturbation	Trained parameter vector after training	Protects the model released by a trusted trainer	Requires a valid sensitivity bound for the entire training algorithm
DP-SGD	Clipped per-example gradients during training	Suitable for many machine-learning models, including deep learning	Requires clipping, calibrated noise, and privacy accounting; may reduce accuracy and increase training cost

Table 1: Three common locations for introducing randomness in differentially private machine learning.

9 Research Directions

Several open and active research directions arise when applying differential privacy to machine learning:

- **Utility at strong privacy levels:** design mechanisms that retain predictive accuracy while satisfying small privacy budgets.
- **Efficient privacy accounting:** improve estimates of cumulative privacy loss across many training steps, repeated releases, and composed mechanisms.
- **Privacy for large models:** reduce the computational and accuracy costs of DP training for large neural networks and foundation models.
- **User-level and group privacy:** protect a person's many records or a group of related records rather than a single row.
- **Private data collection and distributed training:** combine local DP, secure aggregation, and distributed optimization to reduce trust in a central collector.
- **Fairness and subgroup utility:** understand how noise affects performance across demographic or data subgroups without weakening the privacy guarantee.
- **Private inference and model confidentiality:** combine DP with cryptographic techniques when a system must also hide a user's query or keep model parameters secret. These are distinct objectives from training-data DP.
- **Auditing and deployment:** build reproducible methods to report privacy budgets, assumptions, adjacency definitions, and accuracy costs in real systems.

10 Summary

Differential privacy formalizes a guarantee that the output of a randomized mechanism changes only within specified bounds when one individual's contribution to the dataset changes. Neighboring datasets define the comparison; ε and δ quantify the guarantee; sensitivity helps determine the noise required. In machine learning, noise may be introduced before training through input perturbation, after training through output perturbation, or during training through DP-SGD. The choice depends on the trust model, the model-training algorithm, the data domain, the desired privacy unit, and the acceptable loss in utility. Noise by itself is not a privacy proof: the mechanism and its parameters must be calibrated and analyzed.

Selected References

1. C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy.* Foundations and Trends in Theoretical Computer Science, 2014. https://www.cis.upenn.edu/~aaroth/Papers/privacybook.pdf

2. K. Chaudhuri, C. Monteleoni, and A. D. Sarwate, "Differentially Private Empirical Risk Minimization," *Journal of Machine Learning Research*, vol. 12, pp. 1069–1109, 2011. https://jmlr.org/papers/v12/chaudhuri11a.html

3. M. Abadi et al., "Deep Learning with Differential Privacy," *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2016. https://arxiv.org/abs/1607.00133

4. J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local Privacy and Statistical Minimax Rates," *Proceedings of the IEEE Symposium on Foundations of Computer Science*, 2013. https://arxiv.org/abs/1302.3203

5. US Census Bureau, "2020 Census and Differential Privacy." https://www.census.gov/data/academy/webinars/2020/2020-census-and-differential-privacy.html